

TP09 - SPE SLAM – Projet Perso

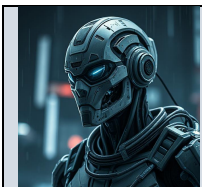
Thème : Concevoir une application Web

Contexte :

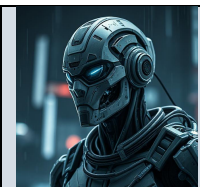
- Formation : BTS SIO (1ère année)
- Durée : 3 séances de 4 heures (2x 2h)
- Objectif pédagogique : Concevoir une application

SOMMAIRE :

I. Mission 1 : Définir précisément votre projet	4
1. Présenter le contexte.....	4
2. Identifier les utilisateurs	4
3. Résumer votre projet	5
II. Mission 2 : Définir les fonctionnalités	6
III. Mission 3 : Définir votre MVP	7
a) Fonctionnalités obligatoires	7
b) Fonctionnalités bonus.....	7
1. CHECKPOINT 1 : Faites valider votre périmètre	8
IV. Mission 4 : Identifier les données	9
V. Mission 5 : Construire le MCD	10
1. Vérifiez votre modèle.....	11
VI. Mission 6 : Passer du MCD au MLD.....	12
VII. Mission 7 : Modéliser les principales classes.....	13
VIII. Mission 8 : Choisir l'architecture technique	14
1. Dessinez votre architecture	15
IX. Mission 9 : Vous devez utiliser une API REST	17
2. CHECKPOINT 2 : Votre projet existe maintenant sur le papier	18
X. Mission 10 : Créer le projet et le dépôt Git.....	19
1. Initialisez Git	20
2. Créez le premier commit	20
3. Créez le dépôt GitHub.....	21
XI. Mission 11 : Écrire le README	22
XII. Mission 12 : Préparer votre plan de développement	23
XIII. Vous avez terminé ?	25



XIV.	Utilisation de l'IA pendant le TP	26
XV.	Livrables obligatoires en fin de séance	27



TP SLAM : Projet individuel

Séance 1 : De l'idée au projet

Durée : 4 heures

Contexte

Pendant les trois prochaines semaines, vous allez concevoir et développer individuellement une application Web.

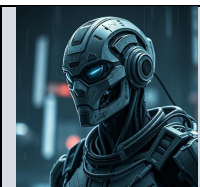
Vous avez choisi ou commencé à choisir votre sujet lors du cours précédent.

Aujourd'hui, votre objectif n'est pas de développer le plus vite possible.

Votre objectif est de répondre à cette question :

Qu'est-ce que je vais développer, pour qui, avec quelles données, quelles fonctionnalités et quelle architecture ?

À la fin de la séance, votre projet doit être suffisamment défini pour que vous puissiez commencer le développement sans improviser son fonctionnement.



I. Mission 1 : Définir précisément votre projet

Temps conseillé : 30 minutes

Créez à la racine de votre projet un dossier :

`docs/`

Puis créez :

`docs/01-expression-besoin.md`

Ce document va décrire votre projet.

1. Présenter le contexte

Expliquez en quelques lignes :

- Quel problème votre application cherche à résoudre ;
- Qui rencontre ce problème ;
- Pourquoi votre application peut être utile.

Exemple :

Je possède plusieurs jeux vidéo que je prête régulièrement. Je souhaite développer une application permettant d'enregistrer ma collection et de savoir quels jeux sont actuellement prêtés et à qui.

Ne décrivez pas encore la solution technique.

Évitez : « Je vais faire une application React avec Spring Boot et PostgreSQL. »

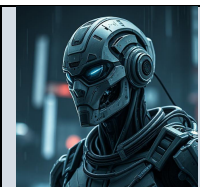
À ce stade, on parle du besoin, pas de la technologie.

2. Identifier les utilisateurs

Indiquez qui utilisera votre application.

Exemple :

Utilisateur principal :
Propriétaire de la collection.



Il doit pouvoir :

- consulter sa collection ;
- ajouter un jeu ;
- rechercher un jeu ;
- enregistrer un prêt ;
- enregistrer un retour.

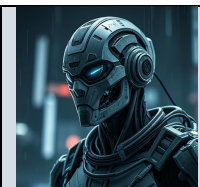
Votre application peut avoir plusieurs types d'utilisateurs, mais n'en inventez pas uniquement pour complexifier le projet.

3. Résumer votre projet

Complétez :

Mon application permet à _____ de _____ afin de _____.

Cette phrase doit permettre à quelqu'un qui ne connaît pas votre projet de comprendre immédiatement son objectif.



II. Mission 2 : Définir les fonctionnalités

Temps conseillé : 30 minutes

Complétez le document :

`docs/01-expression-besoin.md`

Ajoutez une section :

`## Fonctionnalités`

Listez maintenant ce que l'utilisateur pourra réellement faire.

Utilisez la formulation :

`L'utilisateur doit pouvoir...`

Par exemple :

`L'utilisateur doit pouvoir consulter les jeux.`

`L'utilisateur doit pouvoir ajouter un jeu.`

`L'utilisateur doit pouvoir modifier un jeu.`

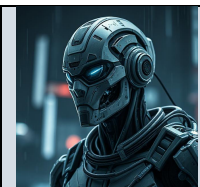
`L'utilisateur doit pouvoir supprimer un jeu.`

`L'utilisateur doit pouvoir rechercher un jeu.`

`L'utilisateur doit pouvoir enregistrer le prêt d'un jeu.`

`L'utilisateur doit pouvoir enregistrer le retour d'un jeu.`

Essayez d'identifier environ 5 à 10 fonctionnalités principales.



III. Mission 3 : Définir votre MVP

Temps conseillé : 20 minutes

Vous avez trois semaines.

Vous n'allez donc probablement pas pouvoir développer toutes les idées que vous avez en tête.

Vous allez définir votre : **MVP : Minimum Viable Product**

C'est la version minimale de votre application qui répond réellement au besoin.

Créez :

`docs/02-mvp.md`

Créez deux catégories.

a) Fonctionnalités obligatoires

Exemple :

MVP

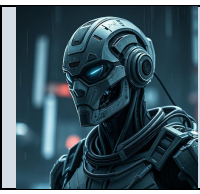
- ☐ afficher les jeux
- ☐ ajouter un jeu
- ☐ modifier un jeu
- ☐ supprimer un jeu
- ☐ rechercher un jeu
- ☐ enregistrer un prêt

b) Fonctionnalités bonus

BONUS

- ☐ statistiques
- ☐ images des jeux
- ☐ authentification
- ☐ mode sombre
- ☐ export PDF
- ☐ notifications

La règle pendant les trois semaines sera : **MVP avant BONUS.**



Une application avec six fonctionnalités simples qui fonctionnent est préférable à une application avec quinze fonctionnalités dont dix sont cassées.

1. CHECKPOINT 1 : Faites valider votre périmètre

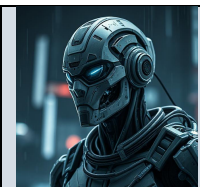
Avant de continuer, présentez rapidement (2min de pitch) :

- ☐ votre sujet
- ☐ votre besoin
- ☐ vos utilisateurs
- ☐ vos fonctionnalités
- ☐ votre MVP

L'objectif est notamment de vérifier que votre projet n'est pas :

trop petit <—— raisonnable ——> beaucoup trop ambitieux

Une fois le périmètre validé par moi ^^, continuez.



IV. Mission 4 : Identifier les données

Temps conseillé : 30 minutes

Vous savez maintenant ce que fait votre application.

Vous devez déterminer quelles informations elle doit mémoriser.

Prenons une application de prêt de jeux.

On pourrait identifier :

JEU

id
titre
plateforme
genre

puis :

PERSONNE

id
nom
email

et :

PRET

id
datePret
dateRetour

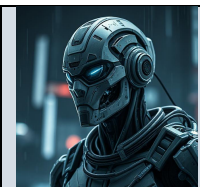
Vous venez de commencer à identifier les entités de votre application.

Pour votre propre projet, listez :

1. Les entités ;
2. Leurs propriétés ;
3. Leurs relations.

Ne pensez pas encore aux tables SQL.

Pensez au monde réel.



V. Mission 5 : Construire le MCD

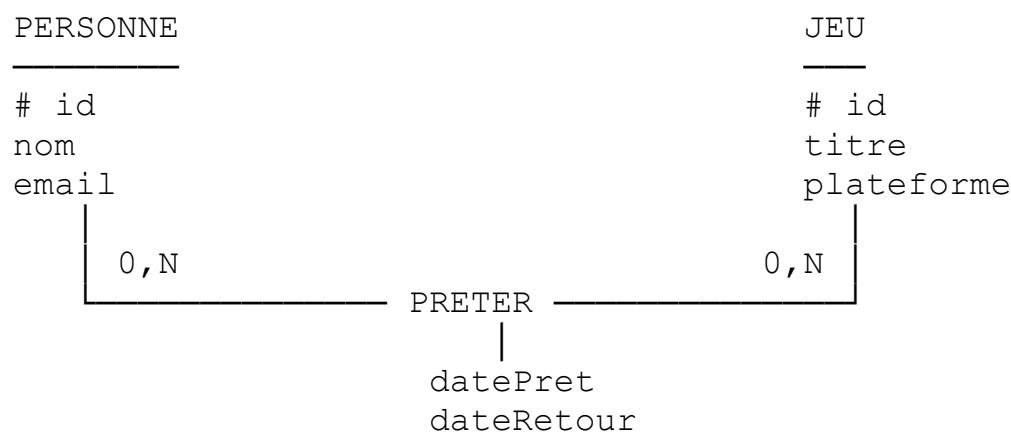
Temps conseillé : 35 minutes

Réalisez maintenant le Modèle Conceptuel de Données de votre application.

Votre MCD doit faire apparaître :

- Les entités ;
- Les attributs principaux ;
- Les identifiants ;
- Les associations ;
- Les cardinalités.

Exemple simplifié :



Vous pouvez utiliser l'outil de modélisation de votre choix.

Exportez ensuite votre diagramme dans :

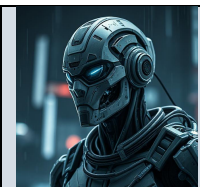
docs/

par exemple :

docs/mcd.png

ou :

docs/mcd.pdf



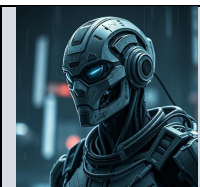
1. Vérifiez votre modèle

Pour chaque relation, essayez de la lire en français.

Par exemple :

- ➔ Une personne peut emprunter zéro ou plusieurs jeux.
- ➔ Un jeu peut être prêté plusieurs fois au cours du temps.

Si la phrase obtenue n'a aucun sens, vos cardinalités sont probablement à revoir.



VI. Mission 6 : Passer du MCD au MLD

Temps conseillé : 20 minutes

Transformez maintenant votre MCD en Modèle Logique de Données.

Exemple :

PERSONNE

id_personne PK

nom

email

JEU

id_jeu PK

titre

plateforme

PRET

id_pret PK

date_pret

date_retour

id_personne FK

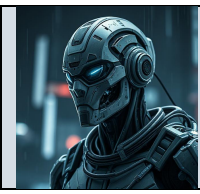
id_jeu FK

Enregistrez-le dans :

docs/03-mld.md

ou exportez votre diagramme dans docs/.

À ce stade, vous devez pouvoir répondre à : quelles tables ma base de données contiendra-t-elle ?



VII. Mission 7 : Modéliser les principales classes

Temps conseillé : 25 minutes

Votre base de données représente les données.

Votre application devra également manipuler ces données dans le code.

Réalisez un diagramme de classes simplifié.

Il n'est pas nécessaire de modéliser toute l'application.

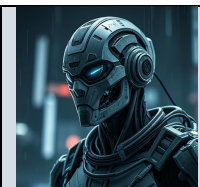
Concentrez-vous sur les principales classes métier.

Exemple :

Jeu
id titre plateforme genre
emprunter() restituer()

Vous pouvez également représenter les relations entre les classes.

Enregistrez le résultat dans : docs/diagramme-classes.png ou équivalent.



VIII. Mission 8 : Choisir l'architecture technique

Temps conseillé : 25 minutes

Maintenant seulement, vous allez réellement choisir vos technologies.

Créez :

`docs/04-architecture.md`

Complétez :

`# Architecture technique`

`## Frontend`

Technologie :

Pourquoi ce choix :

`## Backend`

Technologie :

Pourquoi ce choix :

`## Base de données`

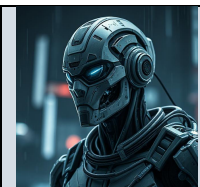
Technologie :

Pourquoi ce choix :

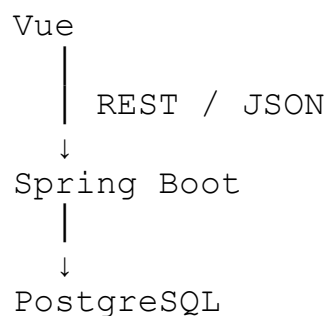
`## Communication frontend/backend`

Méthode :

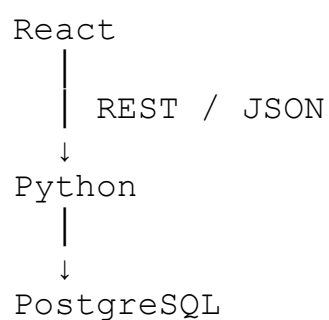
Plusieurs architectures sont possibles.



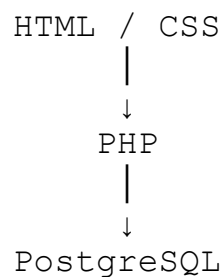
Par exemple :



ou :



ou encore une application PHP plus classique :

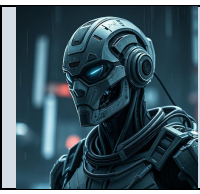


Vous n'avez aucune obligation de choisir l'architecture la plus complexe.

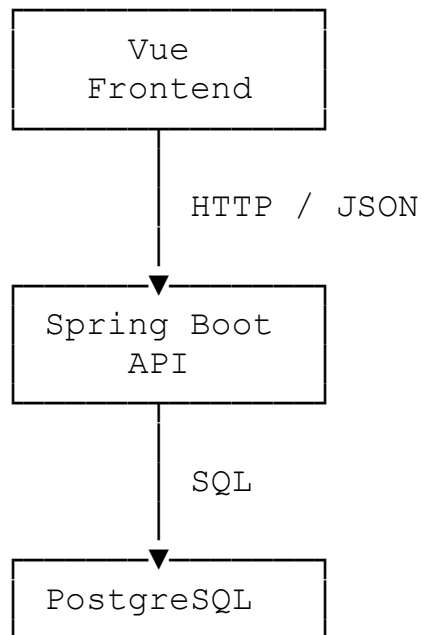
Choisissez une architecture que vous pensez pouvoir terminer et expliquer.

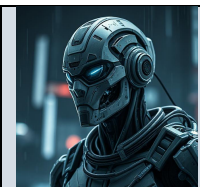
1. Dessinez votre architecture

Ajoutez un petit schéma.



Exemple :





IX. Mission 9 : Vous devez utiliser une API REST

Temps conseillé : 20 minutes

Cette mission concerne les projets utilisant une architecture frontend/backend avec API.

Avant de coder votre API, définissez son contrat.

Créez :

`docs/05-api.md`

Prenons une ressource jeux.

```
GET      /api/jeux
GET      /api/jeux/{id}
POST     /api/jeux
PUT      /api/jeux/{id}
DELETE   /api/jeux/{id}
```

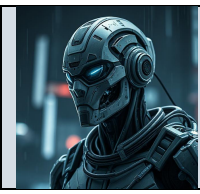
À vous de définir les principales routes de votre application.

Par exemple :

Méthode	Route	Fonction
GET	/api/jeux	Récupérer tous les jeux
GET	/api/jeux/{id}	Récupérer un jeu
POST	/api/jeux	Ajouter un jeu
PUT	/api/jeux/{id}	Modifier un jeu
DELETE	/api/jeux/{id}	Supprimer un jeu

Ne développez pas encore les routes.

Vous définissez d'abord le contrat que votre application devra respecter.

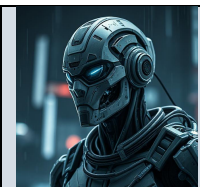


2. CHECKPOINT 2 : Votre projet existe maintenant sur le papier

Avant de passer à l'installation, vérifiez :

- ☐ besoin défini
- ☐ utilisateurs identifiés
- ☐ fonctionnalités définies
- ☐ MVP défini
- ☐ entités identifiées
- ☐ MCD réalisé
- ☐ MLD réalisé
- ☐ diagramme de classes réalisé
- ☐ architecture technique choisie
- ☐ API définie

Si plusieurs éléments ne sont pas terminés, ne commencez pas encore à développer.



X. Mission 10 : Créer le projet et le dépôt Git

Temps conseillé : 10 minutes

Vous pouvez maintenant préparer votre environnement.

Créez votre dossier projet.

Exemple :

```
mkdir gestion-jeux  
cd gestion-jeux
```

Votre structure dépendra ensuite de votre architecture.

Exemple frontend/backend :

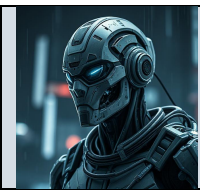
```
gestion-jeux/  
├── frontend/  
├── backend/  
├── docs/  
├── .gitignore  
└── README.md
```

Ou pour une application plus simple :

```
gestion-jeux/  
├── src/  
├── docs/  
├── .gitignore  
└── README.md
```

N'ajoutez pas des dossiers uniquement parce qu'ils apparaissent dans cet exemple.

Votre arborescence doit correspondre à votre propre architecture.



1. Initialisez Git

À la racine :

```
git init -b main
```

Vérifiez :

```
git status
```

Créez votre `.gitignore` avant d'envoyer votre projet.

Attention notamment à :

```
node_modules/  
venv/  
target/  
.env
```

selon votre technologie.

Ne placez jamais dans Git :

```
mot de passe  
clé API  
token  
secret
```

2. Créez le premier commit

Ajoutez votre documentation :

```
git add .
```

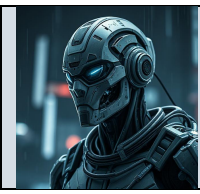
Puis :

```
git commit -m "Initialisation et conception du projet"
```

Remarquez quelque chose d'important :

Votre premier commit contient déjà du travail alors que vous n'avez écrit aucune ligne de code.

L'analyse et la conception font partie du projet.



3. Créez le dépôt GitHub

Créez un dépôt vide correspondant à votre projet.

Puis reliez-le :

```
git remote add origin URL_DU_DEPOT
```

Vérifiez :

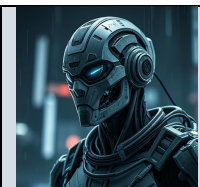
```
git remote -v
```

Puis :

```
git push -u origin main
```

Rechargez la page GitHub.

Votre documentation doit apparaître.



XI. Mission 11 : Écrire le README

Temps conseillé : 15 minutes

Votre fichier :

README.md

doit au minimum contenir :

Nom du projet

Présentation

Quelques lignes présentant l'application.

Objectif

Quel problème l'application cherche-t-elle à résoudre ?

Fonctionnalités principales

- ...
- ...
- ...

Technologies envisagées

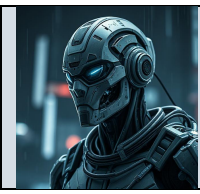
- Frontend :
- Backend :
- Base de données :

État du projet

Analyse et conception en cours.

Le README évoluera pendant les trois semaines.

Il ne doit donc pas devenir un document oublié créé uniquement aujourd'hui.



XII. Mission 12 : Préparer votre plan de développement

Temps conseillé : 15 minutes

Avant de coder, posez-vous une dernière question : **dans quel ordre vais-je développer mon application ?**

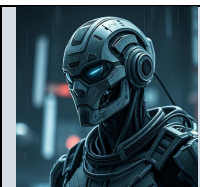
Évitez :

authentification
↓
dark mode
↓
logo
↓
animation
↓
base de données

Commencez par le cœur du projet.

Exemple :

1. Créer la base de données
2. Créer les modèles / entités
3. Afficher les données
4. Ajouter une donnée
5. Modifier une donnée
6. Supprimer une donnée
7. Ajouter la recherche
8. Ajouter les autres fonctionnalités du MVP
9. Tester
10. Améliorer l'interface
11. Bonus éventuels



Créez :

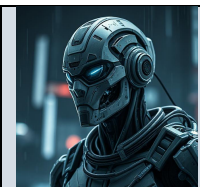
`docs/06-plan-developpement.md`

Transformez votre plan en checklist :

Plan de développement

- [] Créer la base
- [] Créer les entités
- [] Afficher les données
- [] Ajouter une donnée
- [] Modifier une donnée
- [] Supprimer une donnée
- [] Ajouter la recherche
- [] Tester le MVP

Vous pourrez cocher les éléments pendant les prochaines séances. Vous pouvez également vous en servir pour définir vous-mêmes vos sprints pour les prochaines séances.



XIII. Vous avez terminé ?

GG.

Mais avant de coder, vérifiez une dernière fois :

Mon projet est-il clairement défini ?

Mon MVP est-il réaliste ?

Mon MCD correspond-il réellement à mon besoin ?

Mon architecture est-elle adaptée ?

Suis-je capable d'expliquer chacun de mes choix ?

Si oui : **Vous avez l'autorisation de commencer le développement avec ma bénédiction.**

Mais commencez par le cœur du projet.

Pas par le CSS. Pas par le logo. Pas par le dark mode. :)

Premier objectif :

APPLICATION

↓

BASE DE DONNÉES

↓

PREMIÈRES DONNÉES

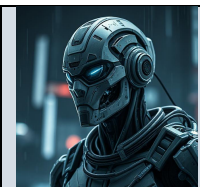
↓

AFFICHAGE

Effectuez ensuite un commit dès qu'une première étape significative fonctionne.

Exemple :

```
git add .  
git commit -m "Création du modèle de données"  
git push
```



XIV. Utilisation de l'IA pendant le TP

L'IA est autorisée.

Vous pouvez notamment lui demander :

« Voici mon expression du besoin. Est-ce que certaines fonctionnalités sont ambiguës ? Ne m'en ajoute pas de nouvelles. »

ou :

« Voici mon MCD. Vérifie la cohérence des cardinalités et explique-moi les problèmes éventuels. »

ou :

« Voici mon architecture. Explique-moi les avantages et les difficultés de ce choix pour un projet réalisable en trois semaines. »

En revanche, évitez :

« Fais toute l'analyse et le MCD de mon application. »

Votre travail consiste précisément aujourd'hui à apprendre à effectuer cette analyse.

Règle à respecter

IA

↓

PROPOSITION

↓

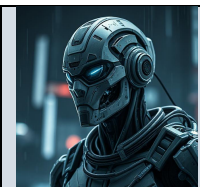
VOTRE ANALYSE

↓

VOTRE DÉCISION

L'IA peut se tromper.

Et surtout : **Vous êtes responsable de tout ce qui apparaît dans votre projet.**



XV. Livrables obligatoires en fin de séance

À la fin des 4 heures, votre dépôt GitHub doit contenir au minimum :

```
mon-projet/  
├── docs/  
│   ├── 01-expression-besoin.md  
│   ├── 02-mvp.md  
│   ├── 03-mld.md  
│   ├── 04-architecture.md  
│   ├── 05-api.md  
│   ├── 06-plan-developpement.md  
│   ├── mcd.png  
│   └── diagramme-classes.png  
├── .gitignore  
└── README.md
```

Les noms exacts peuvent varier selon vos outils et votre projet.

Mais vous devez pouvoir montrer et expliquer :

1. L'expression du besoin ;
2. Les utilisateurs ;
3. Les fonctionnalités ;
4. Le MVP et les bonus ;
5. Le MCD ;
6. Le MLD ;
7. Le diagramme de classes ;
8. L'architecture technique ;
9. Le contrat de l'API si nécessaire ;
10. Le plan de développement ;
11. Le README ;
12. Le dépôt Git/GitHub avec au moins un commit propre.

Votre objectif à 11 h

Vous ne devez pas nécessairement me montrer une application. Vous devez pouvoir me dire :

« Je sais ce que je vais développer, pourquoi je le développe, quelles données je vais utiliser, comment mon application sera organisée et dans quel ordre je vais la construire. »

Si vous êtes capables de répondre clairement à ces questions, les 4 heures ont été productives et vous avez été efficace. Donc GG.